

Contents

SPECIFICATION — Eval Harness for the Chapter-3 RAG Pipeline (golden datasets, regression reports, gates, eval-driven development, + uv)		1
0. Intent and purpose		2
1. Actors and goals		3
2. Requirements (intent, high level)		5
3. Behavior and state model		7
3.1 Lifecycle scope		7
3.2 The eval-time flow (one dataset run)		7
3.3 The report pipeline (artifacts are the interface)		8
4. Interfaces / contracts		8
C-01 Golden dataset types		8
C-02 AoE adapter (pinned ch3 interface)		8
C-03 Evaluator pipeline		9
C-04 Metrics math (pure; zero-denominator guarded, I-001)		9
C-05 <code>eval.json</code> artifact (versioned, R-21)		10
C-06 Compare report (§23)		10
C-07 Gates config (§24/§25)		10
C-08 Failure classifier precedence (§34)		10
C-09 Human labels + judge validation (§26)		10
C-10 Pairwise evaluation (§27, optional)		11
C-11 Production → golden scaffold (§28, optional)		11
5. Interface specification		11
5.1 CLI — primary surface (<code>rag-eval</code> , R-18)		11
5.2 GUI — optional surface (<code>rag-eval-gui</code> , R-16)		11
6. Invariants (must hold in every valid implementation)		12
7. Constraints (precise and measurable)		13
8. Edge cases and failure semantics		13
9. Acceptance criteria, tests, and evals		14
9.1 Dataset (C-01, R-02/R-13)		14
9.2 Pipeline end-to-end (C-02/C-12, R-01/R-14)		15
9.3 Metrics (C-04, R-04)		15
9.4 Determinism + gold isolation (R-14/R-20)		15
9.5 Compare (C-06, R-06)		15
9.6 Gates (C-07, R-07)		15
9.7 Failure classification (C-08, R-10)		15
9.8 Deliberate regression, §33 (R-09)		16
9.9 Judge validation (C-09, R-11)		16
9.10 Optional surfaces (R-12/R-13/R-16)		16
9.11 Manual / real-path smoke (opt-in — not part of <code>uv run pytest</code>)		16
10. Dependencies and environment		16
11. Traceability matrix (id → where realized)		16

SPECIFICATION — Eval Harness for the Chapter-3 RAG Pipeline (golden datasets, regression reports, gates, eval-driven development, + uv)

- **Status:** v0.1 — draft for implementation review. Written one section at a time, following the ch1/ch2/ch3 lab SPEC pattern; a SPEC_REVIEW pass (per the `spec-review` skill) is expected before v0.2, exactly as ch3 did.
- **Language:** Python 3.12 | Application-under-evaluation (AoE): ch3 `rag` lab (imported as a path dependency) | Evaluators: deterministic checks + `MockJudge` (offline double) / LLM-

as-judge over Ollama (opt-in real) | Schema: jsonschema | Config: PyYAML | HTTP: httpx
| GUI: PyQt5 (optional)

- **Curriculum source:** curriculum/week1/chapter4.md (§1 Why AI Systems Need Evals, §2 The Evaluation Harness, §3 Golden Datasets, §4 What Makes a Good Evaluation Dataset, §5 Deterministic Tests, §6 LLM-as-Judge, §7 Human Evaluation, §8 Pairwise Evaluation, §9 Rubric-Based Evaluation, §10 Accuracy, §11 Precision/Recall, §12 Groundedness, §13 Relevance, §14 Completeness, §15 Hallucination Rate, §16 Tool-Call Success, §17 Latency, §18 Cost, §19 The Evaluation Vector, §20 Evaluation Is a Dataset Problem, §21 Stratified Evaluation, §22 Regression Testing, §23 Regression Reports, §24 Regression Gates, §25 Don't Over-Automate, §26 Evaluator Validation, §27 Pairwise Model Selection, §28 Production Evaluation, §29 Evals Change Engineering, §30 Evals as the Unit-Test Analog, §31 Evals as the Missing Abstraction, §32 The Chapter 4 Exercise, §33 Deliberately Introduce a Regression, §34 Failure Analysis, §35 The Evaluation Matrix, §36–§39 Central Lesson / Deeper Principle / Checklist / Key Takeaways). The upstream **application under evaluation (AoE)** is the ch3 RAG pipeline from labs/week1/chapter3 (its SPEC is the authoritative ch3 interface reference).
- **Scope of this document:** the *authoritative specification* of the ch4 **evaluation subsystem** — the harness that turns the ch3 RAG application from a manually-inspected demo into a measurable, regression-gated engineering artifact (ch4 §31, §36). It is written to Level 2–3 (structured, mostly executable): behavior, interfaces, invariants, edge cases, and failure semantics are made explicit so an agent (or engineer) can derive implementation **and** verification with minimal inference.
- **Normative language:** MUST, MUST NOT, SHALL, and SHALL NOT are normative. SHOULD denotes a strong recommendation; MAY an optional behavior.
- **Principle:** ch4 §31/§30 — the evaluation suite is the *bridge between probabilistic behavior and engineering discipline*; an eval does not specify an exact output string, it specifies a **behavioral contract** and measures whether the probabilistic system satisfies it ($f(x) \in Y_{\text{acceptable}}$, §37). Requirements express *intent*; this specification *operationalizes* intent into observable behavior plus the conditions under which we know it is correct.

0. Intent and purpose

Chapter 4's central lesson is the mechanism that makes ch2 (context) and ch3 (retrieval/RAG) **engineerable** at all:

Eval-driven development — the loop **Application** → **Evaluation** → **Metrics** → **Failure analysis** → **Application change** → **Evaluation** (ch4 §0). If you cannot evaluate the behavior, you cannot reliably improve the system (§37).

This lab specifies the §32 exercise (*The Chapter 4 Exercise*): build a **small evaluation harness** around the ch3 RAG system, with a **golden dataset** (50–100 cases, §32/§3), a full **evaluation vector** of metrics (§19), **stratified** breakdowns (§21), **regression reports** comparing versions (§23), **regression gates** suitable for CI (§24), **failure attribution** into the §34 taxonomy, and a validated **evaluator** (§26). The six §32 experiments (chunk size, reranking off, k , query expansion, LLM change, context ordering) plus the §33 deliberate regression (**top_k** 5 → 30) are the acceptance criteria of this spec — *run them through the harness, not by eye*.

The harness instantiates ch4 §2's architecture (dataset → application → outputs → evaluator → metrics → regression report) around **one deterministic boundary and one probabilistic boundary**, the same reliability split as ch1/ch2/ch3:

- **Deterministic boundary (pure, offline, reproducible, no LLM, no network):** the **dataset loader/validator**, the **pipeline driver** (which merely wires the AoE), the **evaluator's deterministic checks**, the **metric math** (P@k / R@k reuse of ch3 `metrics.py`, plus groundedness, completeness,

hallucination rate, latency percentiles, cost-per-success), the **compare/gate logic**, the **failure classifier**, and the **report writers**. Offline, the AoE itself runs on ch3’s mock doubles (`MockEmbedder` / `MockLLM` / `MockJudge` / `MockReranker` / `MockQueryExpander`), so the entire evaluation can be re-run by CI without any model (ch3 R-17, carried).

- **Probabilistic boundary (unreliable components):** the AoE’s real Ollama generation (`qwen3.8`) and optionally a **real LLM-as-judge** evaluator. Both are isolated behind interfaces and replaced by deterministic doubles for the automated suite. ch3 R-19’s model-availability taxonomy (`DEGRADED MOCK` / `PULL_REQUIRED` / `RUN_REAL`, E-13) is **carried over unchanged** so that “why did this run degrade to mocks” is never ambiguous in an eval report (F-013).

Evaluation hierarchy discipline (§5–§7). The ch4 §7 hierarchy — deterministic tests → automated metrics → LLM evaluation → human evaluation — is encoded as an *ordered evaluator pipeline*: deterministic checks (schema-validity, citation-chunk membership, structure) run first and *before any judge*; LLM-judge verdicts are gated on those checks; human labels (when present) are used to **validate the judge itself** (§26), not as the primary regression groove.

The evaluation vector (§19) is the reporting object. Per-case and aggregate:

$$\mathbf{Q} = (A, P, R, G, C, H, L, K)$$

where A = correctness (ch3 verdict `correct`), P = precision@k, R = recall@k, G = groundedness (faithfulness = supported claims / total factual claims, ch3 §21), C = completeness (reflected gold facts / total gold facts), H = hallucination rate (unsupported claims / total claims, §15), L = latency (near-rank percentiles $P50/P90/P95/P99$, §17), and K = cost per *successful* case (§18). T (tool success, §16) is a reserved slot — out of scope for the ch3 RAG’s retrieval path (single LLM call, no tool loop) but kept in the vector schema for the agent weeks.

Deployment decision: identical to ch3 R-19 — the real LLM path is **local Ollama** at `http://localhost:11434`; when unavailable the harness **degrades to the mock doubles** and says so with ch3’s exact E-13 banners. The harness’s *own* deterministic core is model-free (import graph scan, T-02 analog of ch3 I-009/R-20).

Relationship to ch3. The AoE is consumed *by import* (path dependency on `labs/week1/chapter3/src`), not re-implemented: ch4 exercises ch3’s `build_index` / `run_case` / `run_dataset` interface exactly as ch3’s own `pipeline.py` does. This is deliberate, and it is the entire pedagogical point of §32: **an evaluation harness wraps an existing application through a stable interface** — you evaluate what you already built. The ch3 index-time vs query-time flag boundary (ch3 §3.1, F-003) is carried explicitly into the experiment toggles (I-008).

Primary product surface: a CLI (`rag-eval`) with subcommands `check` (validate a golden dataset), `run` (execute one evaluation and emit `eval.json` + a human report), `compare` (regression report vs a baseline), `gates` (CI decision: exit 0 / 1), `judge-check` (validate an evaluator against human labels, §26), and two optional surfaces — `pair` (§27 pairwise model selection) and `new-case` (§28 production-failure → golden case scaffolding). An optional PyQt5 GUI (`rag-eval-gui`) browses run reports — never runs inference itself.

1. Actors and goals

Actor	Goals
User (human, single process)	Validate the golden dataset; run the evaluation over the AoE (mock or real); compare two runs (regression report with per-metric Δ); enforce gates in CI (exit 0/1); classify failures by the §34 taxonomy; optionally browse a run report in the GUI. (single-principal — no inter-principal authorization, ch3 F-009 carried; access_level is carried but not consumed.)
Dataset / Generator (dataset.py)	Load the golden dataset (50–100+ EvalCases), validate schema, category membership, and reference closure (every relevant_chunks id exists in the corpus); emit load errors deterministically (ch3 I-013 analog).
AoE adapter (aoe.py)	Drive the ch3 application through its pinned interface (C-02): build_index(...) (index-time flags), run_case(question, index, query_time_flags) → AoEResult (answer, retrieved chunks, scores, raw/parsed output, verdict, usage tokens, latency, trace). Wraps ch3; never re-implements RAG.
Evaluator (evaluator.py: DeterministicChecks first, then MockJudge offline / OllamaJudge real)	Run deterministic structure checks <i>before</i> any judge (§5); produce a per-case verdict + metrics row; classify the failure stage on non-pass (§34).
Metrics (metrics.py)	Reuse ch3 metrics.py for P@k / R@k / MRR / MAP / NDCG; add the §19 evaluation-vector math (groundedness, completeness, hallucination rate, latency percentiles, cost per success). Pure, headless, testable.
Compare / Gates (compare.py)	Load two eval.json artifacts (schema-versioned), emit the §23 regression report (per-metric directional Δ), and evaluate §24 gate thresholds as hard directional constraints — never a weighted composite score (§25).
JudgeValidator (judge_check.py)	Compare a judge’s verdicts against a human-labeled sample (§26): emit agreement rate + confusion pairs — “the measurement system must itself be measured.”
Pairwire (pair.py, <i>optional</i>)	§27 pairwise model selection: run two AoE configs over the same dataset, judge picks a winner per case, emit WinRate .
Ollama daemon (<i>external</i>)	Local runtime at http://localhost:11434 : real generation (/api/chat) and optionally a real judge. Not part of this project; ch3’s E-13 taxonomy (carried) resolves its absence.
FailureClassifier (failure.py)	Map every non-pass case to exactly one §34 taxonomy value (RETRIEVAL_FAILURE / CONTEXT_FAILURE / GENERATION_FAILURE / PARSING_FAILURE / EVALUATION_FAILURE) using the documented precedence; preserve the full trace record (§34).

Actor	Goals
UI (<code>ui.py</code> , <i>optional</i>)	Browse a saved <code>eval.json</code> report offline (per-case traces, regression table, verdict pills); never blocks on inference; offscreen-testable (ch3 R-16 analog).

2. Requirements (intent, high level)

ID	Statement
R-01	The system shall execute the ch4 §2 harness pipeline — Dataset → Application → Outputs → Evaluator → Metrics → Regression Report — where the Application under Evaluation (AoE) is the ch3 RAG pipeline, reached <i>only</i> through the pinned adapter interface (C-02). The harness never re-implements retrieval/generation; it wires, measures, and compares.
R-02	The golden dataset (§3/§32) shall hold 50–100+ <code>EvalCases</code> , each with <code>question</code> , <code>reference_answer</code> , <code>relevant_chunks</code> (ground-truth chunk ids), and <code>category</code> , plus ch3-carried <code>gold_facts</code> for completeness (C-01). <code>dataset check</code> MUST validate schema, category membership in the documented <code>CATEGORY_SET</code> (ch3’s seven failure tiers plus <code>{adversarial, boundary, regression}</code> , §21/§35), and reference closure : every <code>relevant_chunks</code> id MUST exist in the corpus index and every required field MUST be present; violations are deterministic load errors (E-02, ch3 I-013 analog).
R-03	The evaluator (§5/§7 hierarchy) shall run deterministic checks before any judge : answer-schema validity, citation- <code>chunk_id</code> membership in the retrieved context, and any property with an exact spec (e.g. <code>amount >= 0</code> , enum membership) are checked deterministically. A case whose answer fails schema validation is attributed <code>PARSING_FAILURE</code> and is judged on that basis (I-005) — no LLM verdict needed.
R-04	The harness shall compute the §19 evaluation vector per case and in aggregate: correctness <code>A</code> (from the ch3 verdict <code>correct</code>), retrieval <code>P@k</code> and <code>R@k</code> (ch3 <code>metrics.py</code> reuse), groundedness <code>G</code> (faithfulness = supported claims / total factual claims, ch3 §21 formula), completeness <code>C</code> (reflected <code>gold_facts</code> / total <code>gold_facts</code>), hallucination rate <code>H</code> (unsupported claims / total claims, §15), latency <code>L</code> as percentiles <code>P50/P90/P95/P99</code> (near-rank interpolation, §17), and cost <code>K</code> per successful case (§18 formulas: <code>cost_success = sum(input+output tokens × price_table) / successes</code>). §16 tool-success <code>T</code> is a reserved slot (retrieval pipeline has no tool loop in ch3 — kept for the agent weeks).
R-05	Stratification (§21/§35): the aggregate report MUST include a <code>by_category</code> breakdown over the dataset’s declared category set, plus an optional <code>by_difficulty</code> breakdown when the dataset carries difficulty metadata. A global-only aggregate is a report violation (I-012).
R-06	Regression report (§23): <code>compare</code> reads a baseline <code>eval.json</code> and a current <code>eval.json</code> (both schema-versioned, R-21) and emits a per-metric Δ table with a documented direction map (I-004: correctness/groundedness/completeness/P/R are higher-better; hallucination rate/latency percentiles/cost are lower-better). Cells lacking the metric on either side render as the explicit marker <code>n/m</code> (E-07), not a misleading zero.

ID	Statement
R-07	Regression gates (§24): <code>gates</code> evaluates a YAML threshold config (C-07) as hard directional constraints — e.g. <code>accuracy_drop <= 1%</code> , <code>hallucination_increase <= 0.5%</code> , <code>p95_latency_increase <= 20%</code> — and returns exit 0 (pass) or exit 1 (at least one gate fails). A missing metric key on either side fails closed with an explicit message (E-11). Per §25, gates are directional constraints and MUST NOT collapse into a weighted composite score.
R-08	The six §32 experiments SHALL be expressible as flag mappings onto the ch3 CLI surface, honoring the ch3 index-time vs query-time rebuild boundary (ch3 §3.1, F-003): (1) chunk size — index-time, forces <code>build_index</code> before eval; (2) reranking off — query-time; (3) <code>k</code> — query-time; (4) query expansion — query-time; (5) LLM change — <code>--model</code> ; (6) context ordering — index-time. Comparing a baseline against a current whose experiment config implies a <i>stale, mismatched index</i> MUST be refused unless <code>--force-rebuild</code> (I-008, E-08).
R-09	The §33 deliberate-regression exercise (<code>top_k 5 → 30</code>) is a fixed acceptance experiment (T-09): the harness SHALL demonstrate, on the mock path, the documented pattern (recall goes up while precision, groundedness, latency, and cost move adversely) to prove local optimization can degrade the end-to-end system.
R-10	Failure attribution (§34): every non-pass case SHALL be assigned exactly one taxonomy value from the §34 set — <code>RETRIEVAL_FAILURE</code> , <code>CONTEXT_FAILURE</code> , <code>GENERATION_FAILURE</code> , <code>PARSING_FAILURE</code> , <code>EVALUATION_FAILURE</code> — mapped from ch3’s <code>failure_stage</code> via the documented precedence table (C-08, I-006). The full §34 trace record (input, model/prompt version, retrieved docs, retrieval scores, context, raw output, parsed output, evaluator verdict, latency, cost) is preserved per case in <code>eval.json</code> (C-05).
R-11	Evaluator validation (§26): <code>judge-check</code> compares the evaluator’s verdicts against a human-labeled sample file (JSON: <code>case_id → human labels</code>) and emits an agreement rate plus the disagreement pairs. If no human labels exist, the command reports <code>NO_LABELS</code> (E-09) — the harness never silently fabricates agreement.
R-12	Pairwise evaluation (§27, MAY): <code>pair</code> runs two AoE configs over the same dataset, asks the judge for a winner per case, and emits $\text{WinRate}(A) = A_wins / \text{comparisons}$ with per-case verdicts (avoids absolute-score instability, §27).
R-13	Production-loop closure (§28, MAY): <code>new-case</code> scaffolds a valid <code>EvalCase</code> JSON from a stored production <code>AoEResult</code> trace (with <code>relevant_chunks</code> optionally blank-templated) so a production failure can be appended to the golden dataset after a human fills the ground truth. The command never fabricates ground truth (it scaffolds, the human completes).
R-14	Offline determinism (ch3 R-17/R-18 carried): the <i>entire automated suite</i> runs offline via ch3’s mock doubles; the mock path is byte-identical for identical inputs (metrics output formatted to fixed precision, I-002). To keep that invariant while still reporting latency/cost, the mock path synthesizes deterministic surrogates from input/output text lengths (<code>usage_tokens := token estimates</code> , <code>latency_ms := deterministic content-derived</code>) — explicitly labeled synthetic in the report (C-02, E-07). The real Ollama path is opt-in/manual and best-effort.
R-15	Model-availability taxonomy (ch3 R-19/E-13 carried): on <code>--real</code> start the harness resolves <code>DEGRADED MOCK</code> (daemon unreachable → mock + banner, exit 0), <code>PULL_REQUIRED</code> (model not pulled → remediation string + exit 4), or <code>RUN_REAL</code> (pulled → real, no banner, exit 0). Each outcome carries ch3’s exact distinct banner text so a human never misreads why a mock ran.
R-16	An optional PyQt5 GUI (<code>rag-eval-gui</code> , MAY) SHALL browse saved <code>eval.json</code> artifacts offline — run summary, <code>by_category</code> rows, per-case traces, regression Δ tables — without ever running inference (ch3 R-16 analog; offscreen-testable, T-13).

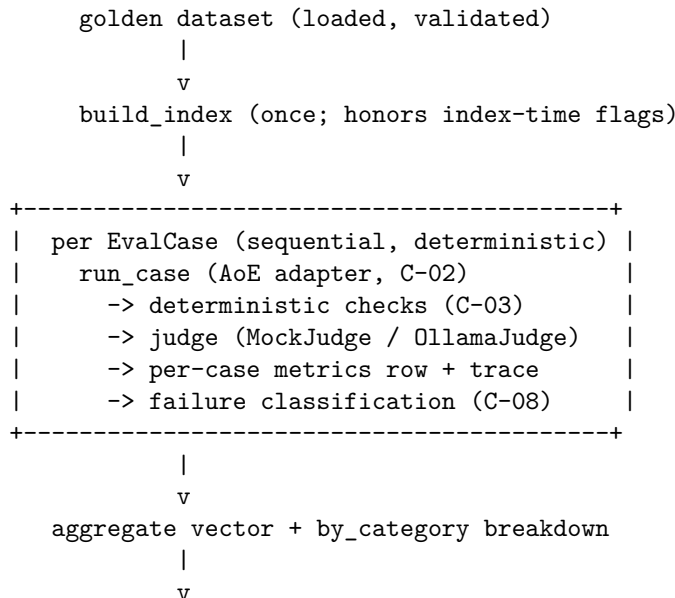
ID	Statement
R-17	The deterministic eval core (<code>dataset.py</code> , <code>evaluator.py</code> , <code>metrics.py</code> , <code>compare.py</code> , <code>gates.py</code> , <code>failure.py</code> , <code>report.py</code>) is LLM- and network-free — asserted by an import/graph source scan (T-02, ch3 I-009/R-20 analog, ch3 T-02).
R-18	The primary product surface is the CLI <code>rag-eval</code> with subcommands <code>check</code> , <code>run</code> , <code>compare</code> , <code>gates</code> , <code>judge-check</code> , plus MAY subcommands <code>pair</code> and <code>new-case</code> (§5.1). All report I/O goes through <code>report.py</code> ; no subcommand prints its own ad-hoc serialization.
R-19	Schema gate (ch3 R-09/R-10/I-010 carried): the <code>eval.json</code> schema and the <code>gates-config</code> schema are validated with <code>jsonschema</code> on every load; the evaluator’s verdict record is validated before metrics are computed (C-03). A malformed artifact failing validation is rejected deterministically (E-06, E-14).
R-20	Gold isolation (ch3 F-001 carried): the AoE’s generation path sees only <code>system/context/question</code> ; the evaluator’s expected values (<code>reference_answer</code> , <code>relevant_chunks</code> , <code>gold_facts</code>) never flow into generation and are consumed <i>only</i> by the evaluator/metrics stage (I-011).
R-21	Report versioning : every <code>eval.json</code> carries a literal <code>eval_report_version == "0.1"</code> field; <code>compare</code> and <code>gates</code> refuse mismatched versions unless <code>--force</code> (E-06). The single literal string is the compatibility surface.

3. Behavior and state model

3.1 Lifecycle scope

The harness has a single execution scope per eval — **eval time** — but it must consciously honor the AoE’s built-in **index-time / query-time** split (ch3 §3.1, F-003). Experiment flags that ch3 classifies as index-time (`--chunk-size`, `--strategy`, `--contextual`, `--embed-model`) force a fresh `build_index` **before** any case runs; query-time flags (`--k`, `--top-n`, `--hybrid`, `--rerank`, `--expand`, `--alpha`, `--model`) recompute on the existing index. The harness carries both classes in one experiment config object; passing a *stale index mismatch* to `compare` is refused unless `--force-rebuild` (R-08, E-08).

3.2 The eval-time flow (one dataset run)



```

eval.json (R-21 versioned)
|
v
compare(v1, v2) -> human report + gates exit 0/1

```

Each EvalCase moves through the deterministic state machine LOADED → INDEXED → RUN → CHECKED → JUDGED → METRIED → CLASSIFIED. A case that fails the deterministic state-machine transition (e.g. its answer JSON fails validation) is marked PARSE_BLOCKED at CHECKED — the JUDGED stage is skipped, verdict.status records PARSE_BLOCKED (I-005), and classification lands on PARSING_FAILURE (I-006 / C-08).

3.3 The report pipeline (artifacts are the interface)

The harness's *only* durable artifacts are files written by `report.py`:

- `dataset_report.json` (from `check`) — validation outcome: ok, violations[] enumerated.
- `eval.json` (from `run`) — the versioned, schema-gated eval artifact (C-05).
- `compare_report.json` + human-readable Δ table (from `compare`).
- `gate_report.json` (from `gates`) — per-gate outcome + aggregate pass/fail (exit code K-03).
- `judge_check_report.json` (from `judge-check`) — agreement rate + disagreement pairs.
- `pair_report.json` (from `pair`, optional) — WinRate + per-case winners.

All downstream consumers (`compare`, `gates`, the GUI) read **only** these artifacts — they never re-run the AoE (I-016).

4. Interfaces / contracts

C-01 Golden dataset types

```

# dataset.py
CATEGORY_SET = {
    "easy", "multi", "chunking", "distractor", "conflict", "recency", "injection",
    "adversarial", "boundary", "regression",      # ch4 §21/§35 extension categories
}

@dataclass
class EvalCase:
    case_id: str          # one golden row; ch3 question row shape extended (§32)
                        # unique; duplicate ids are a load error (E-02)
    question: str
    reference_answer: str
    relevant_chunks: list[str] # ground-truth chunk ids; must be subset of corpus ids (E-02)
    category: str           # must be in CATEGORY_SET (E-02)
    gold_facts: list[str]   # completeness reference (ch3 carried)
    difficulty: str | None = None # optional -> by_difficulty stratification (R-05)
    source: str = "golden"  # "golden" / "production" (§28 filed cases)

@dataclass
class Dataset:
    dataset_id: str        # stable name/hash; compare requires equal ids (E-12)
    cases: list[EvalCase]

```

C-02 AoE adapter (pinned ch3 interface)

```

# aoe.py - the ONLY module allowed to import the ch3 rag package (I-016)
def build_index(corpus_dir: str, index_flags: dict) -> Index: ...
def run_case(case: EvalCase, index: Index, query_flags: dict) -> AoEResult: ...

```

```

@dataclass
class AoEResult:
    question: str
    retrieved_chunks: list[str] # ordered chunk ids (for P@k / R@k / MRR)
    scores: list[float]
    raw_output: str
    parsed_answer: dict | None # None on parse failure -> PARSE_BLOCKED
    verdict: dict # ch3 verdict record (R-19 schema-gated)
    failure_stage: str | None # ch3 stage: retrieval/expansion/reranking/context/generation/judging
    usage_kind: str # "synthetic" (mock) | "measured" (real)
    usage_tokens: int # synthetic: est(context+question)+est(answer); real: counted
    latency_ms: float # synthetic: deterministic content-derived; real: wall clock
    cost_usd: float | None # real only (price_table); synthetic -> None (E-07)

```

The adapter splits `index_flags` vs `query_flags` per ch3 §3.1 (F-003); mock-vs-real resolution follows the ch3 E-13 taxonomy (R-15).

C-03 Evaluator pipeline

```

# evaluator.py
class DeterministicChecks:
    def check(self, aoe_result: AoEResult) -> list[dict] # pass/fail records, deterministic-first

class MockJudge:
    # offline double, ground-truth-derived verdict (ch3 MockJudge)
class OllamaJudge:
    # real LLM-as-judge (§6/§26 - itself validated via judge-check)
# Verdict record (JSON-schema gated, R-19): {correct, supported, complete, unsupported_claims,
#   total_factual_claims, faithfulness, completeness, citation_quality, rationale, status}
# status set: PASS | FAIL | PARSE_BLOCKED (I-005)

```

C-04 Metrics math (pure; zero-denominator guarded, I-001)

```

# metrics.py - reuse ch3 metrics.py for P@k / R@k / MRR / MAP / NDCG; add the §19 vector:
METRIC_KEYS = [
    "accuracy", "precision_at_k", "recall_at_k",
    "groundedness", "completeness", "hallucination_rate",
    "latency_p50", "latency_p90", "latency_p95", "latency_p99",
    "cost_per_success",
]

$$
# accuracy = mean(verdict.correct) (zero cases -> 0, documented)
# precision_at_k = |retrieved_k & relevant| / k (k == 0 -> 0; ch3 carried)
# recall_at_k = |retrieved_k & relevant| / |relevant| (no relevant -> 0, documented)
# groundedness = faithfulness = supported / total_factual_claims (documented zero rule:
#   0 claims -> 1.0, "nothing to contradict", per ch3 zero-denominator rule, E-03)
# completeness = reflected gold_facts / len(gold_facts) (empty -> 1.0, documented)
# hallucination_rate = len(unsupported_claims) / total_factual_claims (0 claims -> 0.0)
# latency_pXX = near-rank percentile over ALL cases (sorted, rank = ceil(p/100 * n)) (§17)
# cost_per_success = sum(cost over successes) / count(successes) (0 successes -> n/m, E-03)

```

Output formatting: floats are rendered to fixed `%.4f` and `by_category` keys sorted lexicographically (I-002 byte-identity).

C-05 eval.json artifact (versioned, R-21)

```
{
  "eval_report_version": "0.1",
  "dataset_id": "golden-v1",
  "usage_kind": "synthetic",
  "judge_role": "mock",
  "capabilities": {"rerank": true, "expand": false, "top_k": 5},
  "cases": [ { "case_id": "q-001", "category": "easy", "verdict": {}, "metrics": {},
    "trace": { "retrieved_chunks": [], "scores": [], "raw_output": "..."} } ],
  "aggregate": { "accuracy": 0.9333, "precision_at_k": 0.7200, "by_category": { "easy": {"accuracy": 0.9333, "precision_at_k": 0.7200} } }
}
```

(Rows shown abridged; the schema in `schemas/eval.json` is authoritative and gated on load, R-19.)

C-06 Compare report (§23)

`compare(baseline: EvalArtifact, current: EvalArtifact)` emits, per metric key, `baseline`, `current`, and Δ computed by the **direction map** (I-004): for higher-better keys $\Delta = \text{current} - \text{baseline}$; for lower-better keys (`hallucination_rate`, `latency_*`, `cost_per_success`) $\Delta = \text{baseline} - \text{current}$. A missing metric renders n/m on that row (E-07), never 0. The human table and JSON are emitted by `report.py` (R-18).

C-07 Gates config (§24/§25)

```
# gates.yml - directional hard constraints; validated by schemas/gates.json on load (R-19)
version: 1
gates:
  - {metric: accuracy,          constraint: "drop",    max_pct_points: 1.0}
  - {metric: groundedness,     constraint: "drop",    max_pct_points: 1.0}
  - {metric: hallucination_rate, constraint: "increase", max_pct_points: 0.5}
  - {metric: latency_p95,      constraint: "increase", max_pct: 20.0}
```

Gate evaluation is per-gate boolean; `aggregate = all(gates)`; exit 0 pass / 1 fail (K-03). MAY gate types: absolute floors/ceilings (`min_value`, `max_value`) for safety-grade metrics per §25 — hard constraints regardless of other deltas. Unknown metric keys are config errors (I-015).

C-08 Failure classifier precedence (§34)

Evaluate in this exact order (I-006):

1. `PARSING_FAILURE` — the AoE answer failed schema validation at `CHECKED`.
2. Label-evidence: `EVALUATION_FAILURE` — present in the human-label disagreement set (and only when such evidence exists; never auto-asserted, E-16).
3. `RETRIEVAL_FAILURE` — `ch3 failure_stage` in `{retrieval, expansion, reranking, chunking}`.
4. `CONTEXT_FAILURE` — `ch3 stage context` (retrieved but omitted from assembled context).
5. `GENERATION_FAILURE` — `ch3 stage generation/judging` (evidence present, answer wrong).
6. Fallback `GENERATION_FAILURE` when no stage is available (the §34 set is total).

C-09 Human labels + judge validation (§26)

```
{"q-001": {"correct": true, "supported": true, "complete": false, "note": "vague on scope"}}
```

`judge-check` computes, per verdict field, `agreement = agreements / (cases with human labels for that field)`; emits disagreement pairs (`case_id`, `field`, `judge-value`, `human-value`). Long-term practice per §26: an agreement below a tracked threshold is an evaluator regression, not an application regression.

C-10 Pairwise evaluation (§27, optional)

`pair(config_a, config_b)` runs both over the identical dataset; for each case the judge emits **winner** in {A, B, TIE}; $\text{WinRate}(A) = A_wins / \text{comparisons}$ — ties count in the denominator, not the numerator (E-15). Per-case verdicts preserved in `pair_report.json`.

C-11 Production → golden scaffold (§28, optional)

`new-case --trace <AoEResult.json> --case-id <id>` emits an `EvalCase` JSON template with **question** pre-filled and **reference_answer** / **relevant_chunks** / **gold_facts** carrying the sentinel "REPLACE_ME"; **check** deliberately treats REPLACE_ME as a violation (a scaffold is not golden until a human completes ground truth, E-02). Deterministic, offline.

5. Interface specification

5.1 CLI — primary surface (rag-eval, R-18)

Subcommand	Behavior	Exit
<code>rag-eval check --dataset <path></code>	Validate the golden dataset (schema, category membership, reference closure, sentinel check); emit <code>dataset_report.json</code> .	0 ok / 3 violations / 2 usage
<code>rag-eval run --dataset <path> [--mock] --out eval.json</code>	Load → <code>build_index</code> → run all cases → <code>eval.json</code> + human summary.	0 (also on DE-GRADED MOCK) / 4 PULL_REQUIRED
<code>rag-eval compare --baseline a.json --current b.json [--force]</code>	§23 Δ report to stdout + <code>compare_report.json</code> .	0
<code>rag-eval gates --baseline a.json --current b.json --config gates.yml</code>	§24 CI decision, per C-07.	0 pass / 1 fail
<code>rag-eval judge-check --labels labels.json --eval eval.json</code>	§26 evaluator agreement vs human labels.	0 / 3 missing labels / 2 usage
<code>rag-eval pair --a config.json --b config.json [--mock]</code>	§27 WinRate (optional, R-12).	0 / 2 usage
<code>rag-eval new-case --trace <AoEResult.json> --case-id <id></code>	§28 scaffold an <code>EvalCase</code> (optional, R-13).	0 / 2 usage

Usage errors (missing flags, bad paths, unknown subcommand) exit 2 consistently (K-01). The `--force-rebuild` flag is reserved for the experiment path (R-08/E-08); `--force` for `compare/gates` bypasses eval-report-version mismatch (E-06). Global: `--self-check` (source-scan for I-016, T-02 analog), `--verbose/--quiet` (loguru level as in ch3), `--model` (real-path judge override).

5.2 GUI — optional surface (rag-eval-gui, R-16)

A PyQt5 window opens a saved `eval.json` or `compare_report.json` from disk (file picker). It shows: run summary banner (`usage_kind`, judge-role mock/real), `by_category` table, per-case trace detail (retrieved chunks, verdict pills, failure-classification string), and a compare Δ table. It never runs inference — the GUI is read-only over artifacts (I-016). Offscreen-rendered in tests via `pytest-qt` (T-13, ch3 R-16/T-16 analog).

6. Invariants (must hold in every valid implementation)

ID	Invariant
I-001	Zero-denominator safety (ch3 carried): no metric divides by an empty denominator; each zero case falls back to its documented value (C-04: accuracy/P/R/cost_per_success -> 0, groundedness/completeness -> 1.0 “nothing to contradict”, hallucination -> 0.0).
I-002	Byte-deterministic output : for identical inputs, harness output artifacts are byte-identical. Floats are rendered at fixed <code>%.4f</code> ; <code>by_category</code> keys are sorted lexicographically; latencies on the mock path are deterministic content-derived surrogates; the real path is opt-in (never in CI).
I-003	Deterministic-first evaluator : deterministic checks (C-03) execute before, and independently of, the judge verdict; a judge is only ever asked about what passes structure. An application answer failing schema validation never reaches the judge (status <code>PARSE_BLOCKED</code>).
I-004	Directionality map (C-06): every metric’s Δ is computed with its declared direction (higher- or lower-better); the map is centralized in <code>compare.py</code> as a dict — no per-call if <code>metric == ...</code> scattered in reports.
I-005	Verdict status totality : every verdict settles in one of <code>{PASS, FAIL, PARSE_BLOCKED}</code> ; a <code>PARSE_BLOCKED</code> verdict still enters failure classification and metrics. No case is silently dropped.
I-006	Failure-classification totality : the §34 set <code>{RETRIEVAL, CONTEXT, GENERATION, PARSING, EVALUATION}</code> covers every non-pass case exactly once, evaluated by the C-08 precedence; the fallback is <code>GENERATION_FAILURE</code> . The classification must not rethrow “unknown” for an omitted ch3 stage.
I-007	Metric key totality : <code>compare/gates</code> operate over the declared <code>METRIC_KEYS</code> list only; any key outside it is a config error (I-015). Gates operate on the <i>same</i> metric names the report emits.
I-008	Index/query flag boundary : experiment flags that ch3 classifies as index-time (<code>--chunk-size</code> , <code>--strategy</code> , <code>--contextual</code> , <code>--embed-model</code>) force a fresh <code>build_index</code> before any case runs; query-time flags recompute on an existing index. Comparing runs across a stale index is refused unless <code>--force-rebuild</code> (E-08).
I-009	Core is LLM/network-free : the deterministic eval core (<code>dataset.py</code> , <code>evaluator.py</code> , <code>metrics.py</code> , <code>compare.py</code> , <code>gates.py</code> , <code>failure.py</code> , <code>report.py</code>) must not import the ch3 <code>rag</code> package, Ollama, or the network — enforced by the source-scan (T-02). Only <code>aoe.py</code> may import ch3 <code>rag</code> (I-016).
I-010	Schema gate on load : <code>eval.json</code> , <code>compare_report.json</code> , and <code>gates.yml</code> (via <code>pyyaml</code> → <code>jsonschema</code>) are validated on every read before use (R-19); a malformed artifact is a deterministic load error (E-06/E-14).
I-011	Gold isolation : <code>reference_answer</code> / <code>relevant_chunks</code> / <code>gold_facts</code> are consumed by <code>evaluator.py</code> + <code>metrics.py</code> only; they never reach <code>aoe.py</code> (and therefore the generation prompt) — ch3 F-001 carried.
I-012	Stratification totality : aggregate output always includes <code>by_category</code> ; the dataset must therefore declare category labels (§35). Compare never drops stratification; a baseline lacking it is deprecated schema (E-06).
I-013	Dataset integrity (ch3 I-013 carried): corrupt/missing/partial dataset file → deterministic violations enumerated in <code>check</code> ; never a partial load (E-02).
I-014	GUI read-only (ch3 analog): the GUI may only open artifacts; it never blocks on or performs inference (T-13).
I-015	Config validation totality : unknown gate metric keys or malformed YAML fail at load with an explicit config error (not silently ignored), and only <code>METRIC_KEYS</code> names are accepted.

ID	Invariant
I-016	Adapter boundary: the <i>only</i> module permitted to import the <code>ch3 rag</code> package is <code>aoe.py</code> ; <code>compare</code> , <code>gates</code> , the GUI, and report reading open artifact files only (they never re-run the AoE). Enforced by the source-scan (T-02, ch3 I-009/T-02 analog).
I-017	No unseen re-verdict / label fabrication: <code>EVALUATION_FAILURE</code> may be asserted ONLY when a human-label disagreement exists (C-08/E-16); the harness never fabricates evaluation disagreement. Similarly new-case scaffolds (never completes) ground truth (C-11).

7. Constraints (precise and measurable)

ID	Constraint
K-01	Usage-error exit codes: all CLI subcommands exit 2 on usage errors (missing flag, unparsable YAML, missing file); never 0 or 1.
K-02	Offline mock run time (ch3 analog): a full 100-case mock dataset run completes under 5 minutes on the host (CI soft target).
K-03	Gates aggregate exit: <code>gates</code> exit 0 iff all per-gate booleans pass; exit 1 otherwise. <code>gate_report.json</code> is emitted either way.
K-04	Compare/gates output coupling: the stdout Δ table and <code>compare_report.json</code> are emitted by one <code>report.py</code> call so the on-screen text and on-disk JSON cannot disagree.
K-05	Percentile method: latency percentiles use the near-rank method (sorted array, <code>rank = ceil(p/100 * n)</code>), cross-checked in T-07 against simple cases; no interpolation approximations.

8. Edge cases and failure semantics

ID	Case	Semantics
E-01	Corpus path missing/unreadable at <code>run</code>	exit 2 usage error; no partial index (ch3 E-01 carried).
E-02	Dataset has duplicate <code>case_ids</code> / bad category / dangling chunk reference / <code>REPLACE_ME</code> sentinel / bad JSON	<code>check</code> enumerates ALL violations in <code>dataset_report.json</code> , exit 3; a partial load is never accepted (I-013).
E-03	Zero <code>gold_facts</code> , zero unsupported claims, zero total claims, zero corpus successes	documented zero-denominator fallbacks per I-001 (groundedness/completeness become 1.0 “nothing to contradict”; hallucination 0.0; cost n/m).
E-04	AoE answer JSON fails schema validation	<code>verdict.status = PARSE_BLOCKED</code> ; classification <code>PARSING_FAILURE</code> (I-005/I-006); judge not asked.
E-05	AoE run raises (<code>run_case</code> exception)	caught, verdict <code>FAIL</code> status <code>RUN_ERROR</code> , classified <code>GENERATION_FAILURE</code> fallback, harness continues other cases (E-11-follow), warning recorded in trace.

ID	Case	Semantics
E-06	<code>eval.json</code> version mismatch / bad schema in <code>compare / gates</code>	rejected with explicit message (exit 2) unless <code>--force</code> ; schema gate always first (I-010).
E-07	Metric absent on one side of <code>compare / gates</code>	Δ row renders explicit <code>n/m</code> marker; <code>gates</code> treats missing metric as fail-closed (fail, explicit <code>missing-metric</code> string) (I-007).
E-08	Stale index mismatch across <code>compare</code> (experiment flags imply different index)	refused with explicit <code>stale index</code> message unless <code>--force-rebuild</code> (I-008); no silent cross-index compare.
E-09	<code>judge-check</code> and no human-label file matches <code>--labels</code>	exit 3, <code>NO_LABELS</code> report; not auto-agreement (I-017).
E-10	Human-label file empty or wrong shape	<code>check</code> violations enumerated (same as E-02), exit 3; labels must specify at least one of the verdict fields.
E-11	Gates config missing metric key / <code>n/m</code> cell reached	that gate fails closed with explicit <code>missing-metric</code> (I-007/E-07); the overall aggregate reflects it.
E-12	<code>compare</code> or <code>gates</code> invoked with different <code>dataset_ids</code>	rejected in usage error (exit 2); comparing different datasets is a deterministically-detected error.
E-13	<code>--real</code> with Ollama unavailable $\rightarrow$ taxonomy	<code>DEGRADED MOCK</code> (exit 0, banner) / <code>PULL_REQUIRED</code> (exit 4, non-numeric remediation) / <code>RUN_REAL</code> (exit 0) — <code>ch3</code> taxonomy carried verbatim (R-15).
E-14	Report schema drift (e.g. hand-edited <code>n/m</code> becomes numeric 0 in artifact)	schema gate on load detects and rejects (I-010) — <code>n/m</code> is a documented literal, not a zero.
E-15	<code>pair</code> ties	ties count in <code>comparisons</code> denominator, never in the <code>A_wins</code> numerator (C-10).
E-16	Classifier receives an omitted <code>ch3</code> <code>failure_stage</code>	falls back to <code>GENERATION_FAILURE</code> (I-006); never asserts <code>EVALUATION_FAILURE</code> without label evidence (I-017).
E-17	GUI opened without a valid artifact path	shows the open-file dialog; a malformed artifact yields an inline schema error message, never a crash (I-014).
E-18	<code>new-case --trace</code> points at a malformed artifact	schema-gate rejects it before scaffolding, usage error exit 2 (I-010). The scaffold is always valid-JSON.

9. Acceptance criteria, tests, and evals

All subsections below (T-01..T-24) run fully offline under `uv run pytest` (R-14, `ch3` carried); the real Ollama path is §9.11 manual-only. Test ids use `ch3`'s naming discipline: each acceptance row is a **T-NN** id registered in §11.

9.1 Dataset (C-01, R-02/R-13)

- **T-01** check on a valid 5-row dataset exits 0 and produces a valid `dataset_report.json` (schema-gated).
- **T-01b** dataset with a duplicate `case_id` $\rightarrow$ `check` exit 3 + enumerated violation.
- **T-01c** dataset with `REPLACE_ME` sentinel (scaffold) $\rightarrow$ violation (the scaffold is not golden, C-11).

- **T-15** corrupt JSON input → `check` enumerates schema violations; exit 3 (ch3 I-013 analog).

9.2 Pipeline end-to-end (C-02/C-12, R-01/R-14)

- **T-03** `run --mock --out eval.json` over the 5-row dataset → emits valid `eval.json` (R-21), each case carries `verdict` + `metrics` + `failure_classification`, report version 0.1.
- **T-13** GUI offscreen opens the emitted `eval.json` without error (R-16/I-014).
- **T-14** source/structure scan: no LLM/Ollama/network import in the eval core; only `aoe.py` imports `ch3 rag` (I-009/I-016).

9.3 Metrics (C-04, R-04)

- **T-04** P/R@k threshold example over fixed retrieved/relevant lists (hand-computed).
- **T-05** percentile near-rank on a sorted 20-sample latency list → T-05 asserts P50/P95 match K-05's formula.
- **T-05b** zero-denominator fallbacks honored per I-001 (empty gold-facts, no-claims case, zero successes → $n/m/1.0/0.0$ per I-001).
- **T-07** `by_category` aggregation equals per-category arithmetic mean of case metrics (I-012).
- **T-08a** ch3 metric ancestry preserved: retrieval P@k/R@k reuse ch3 `metrics.py` (no rewrite, T-02 covers the boundary).

9.4 Determinism + gold isolation (R-14/R-20)

- **T-06** repeated `run --mock` on byte-identical inputs produces byte-identical `eval.json` (I-002) with fixed `%.4f`.
- **T-06b** corpus one-row parse-error simulation via `ModelAnswer` factory (verdict status `PARSE_BLOCKED`; never spawn an LLM) (I-003/I-005).
- **T-11** gold-isolation test: assert `run_case` receives only `context/system/question` (no reference fields) (I-011).

9.5 Compare (C-06, R-06)

- **T-08** fabricated two-version pair asserts Δ signs on the direction map (higher- vs lower-better both) (I-004).
- **T-08b** `by_category` Δ row for each category appears in output (I-012).
- **T-12** dataset-id mismatch `compare` → exit 2 + explicit message (E-12).

9.6 Gates (C-07, R-07)

- **T-09i** all-pass gates config → exit 0, `gate_report.json` enumerates per-gate booleans.
- **T-09f** config where one gate fails → exit 1 with explicit failing-gate list (K-03).
- **T-09m** missing metric → fail-closed exit 1 + explicit missing-marker (E-11/I-007).
- **T-15b** malformed YAML/unknown metric key → usage error exit 2 (I-015/K-01).

9.7 Failure classification (C-08, R-10)

- **T-10a** schema-invalid answer → `PARSING_FAILURE` classification.
- **T-10b** ch3 stage `expansion` → `RETRIEVAL_FAILURE` (mapping rule C-08 step 3).
- **T-10c** ch3 stage `generation` → `GENERATION_FAILURE` (C-08 step 5).
- **T-10d** label-evidence → `EVALUATION_FAILURE` asserted only when the label disagreement exists (I-017).
- **T-10e** omitted stage → fallback `GENERATION_FAILURE` (I-006).

9.8 Deliberate regression, §33 (R-09)

- **T-09** the fixed §33 exercise is preloaded under `tests/fixtures/` as two artifacts (`top_k=5` vs `top_k=30`); Δ shows **precision/groundedness worse** and **recall better**; documented example (acceptance).

9.9 Judge validation (C-09, R-11)

- **T-17** fabricated labels + eval with one disagreement $\rightarrow$ agreement = computed fraction (not 1.0), disagreement pair listed (I-017 never fabricates).
- **T-08a-reuse** ch3 verdict-shape is preserved (schemas match ch3 verdict fields exactly, R-19).

9.10 Optional surfaces (R-12/R-13/R-16)

- **T-16/T-22** pair twins A/B over 3 fabricated cases $\rightarrow$ `WinRate` correct and ties excluded from numerator (E-15).
- **T-23** `new-case` on a valid trace emits sentinel-filled `EvalCase` schema-valid (and `check flags` the sentinel) (C-11/E-02).
- **T-13** GUI offscreen-reads the pair report too (I-014).

9.11 Manual / real-path smoke (opt-in — not part of `uv run pytest`)

- M-01 with Ollama up and model pulled, `run --real` exits 0, banner `RUN_REAL`, `usage_kind` is measured, real percentiles included.
- M-02 Ollama down $\rightarrow$ `DEGRADED MOCK` exits 0 with ch3 banner; `PULL_REQUIRED` exits 4 with remediation line (R-15/E-13).

10. Dependencies and environment

Python **3.12** via `uv` (ch3 carried); libraries: `pyyaml` (gates YAML), `jsonschema` (schema gate R-19), `loguru` (logging), `httpx` (Ollama real path), plus ch3 `rag` package **as a path dependency** (read-only): `file:${PROJECT_ROOT}/labs/week1/chapter3/src/rag`. Optional GUI: `PyQt5`, `pytest-qt` (R-16). Dev: `pytest`. No network, no Ollama, no model required for CI (R-14).

The ch3 **host prerequisite** is *optional* (needed only for the manual §9.11 real path):

```
ollama pull nomic-embed-text      # embedder    (real path only)
ollama pull qwen3.8:27b-mlx      # generation + judge (real path only)
```

Without them the harness *must* degrade to the mock doubles with ch3's exact banner text (R-15/E-13).

11. Traceability matrix (id $\rightarrow$ where realized)

S0 / §31 thesis	--> pipeline loop in §3.2	--> T-03
R-01 (§2 harness)	--> pipeline wiring (dataset -> aoe -> evaluator -> metrics)	--> T-03
R-02 / R-13 / I-013	--> dataset.py loader + closure validator	--> T-01, T-15
R-03 / I-003 (§5)	--> evaluator.py deterministic-first ordering	--> T-06b, T-11
R-04 (§19 vector)	--> metrics.py accuracy/P/R/G/C/H/L/K math	--> T-04, T-05, T-05b, T-05c
R-05 / I-012 (§21/§35)	--> metrics.py by_category + compare.py preserved	--> T-07, T-08b
R-06 / I-004 (§23)	--> compare.py direction map + n/m	--> T-08, T-08b
R-07 / K-03 (§24/§25)	--> gates.py hard directional constraints	--> T-09i, T-09f
R-08 / I-008 (§32)	--> experiment flags index/query split (from ch3)	--> T-14 scan + E-08
R-09 (§33 deliberate)	--> tests/fixtures top_k 5 vs 30 pair	--> T-09
R-10 / I-005/I-006 (§34)	--> failure.py precedence classifier	--> T-10a..T-10e

R-11 (§26)	--> judge_check.py agreement	--> T-17
R-12 (§27 MAY)	--> pair.py WinRate	--> T-16/T-22
R-13 (§28 MAY)	--> new-case scaffold (REPLACE_ME sentinel)	--> T-01c, E-02
R-14 (offline)	--> mock doubles via ch3; deterministic surrogates	--> T-06, T-03, R-17
R-15 (E-13 carried)	--> aoe.py availability resolution	--> M-02 (§9.11)
R-16 / I-014 (GUI)	--> ui.py read-only artifact browser	--> T-13
R-17 / I-009 (core)	--> source-scan assertion	--> T-14 (ch3 T-02 anal
R-18 (CLI)	--> cli.py subcommands	--> T-03, T-12, T-15b
R-19 / I-010 (schema)	--> schemas.py gate on eval/compare/gates/labels	--> T-01, T-15, E-14
R-20 / I-011 (gold)	--> aoe.py prompt fields only	--> T-11
R-21 (versioning)	--> report.py eval_report_version literal + gate	--> T-03, E-06
I-001 (zero-div)	--> metrics.py documented fallbacks	--> T-05b
I-002 (byte-ident)	--> report.py fixed %.4f + sorted keys	--> T-06
I-004 (direction map)	--> compare.py centralized	--> T-08
I-007 (metric keys)	--> gates.py METRIC_KEYS filter + E-07/E-11	--> T-09m, T-15b
I-015 (config)	--> gates.py YAML validation + unknown keys	--> T-15b
I-017 (labels)	--> failure.py + judge_check.py label evidence	--> T-10d, T-17, E-16
I-016 (adapter)	--> aoe.py boundary (only module importing ch3)	--> T-14
K-01..K-05	--> cli.py exits / metrics near-rank percentiles	--> T-05, T-12, T-15b
E-01..E-18	--> failure.py + cli.py + UI schema-error paths	--> T-03, T-10a..e, T-1
